

Stored Procedure

1. Visualizzare gli studenti iscritti a un corso (dato l'ID)

```
DELIMITER //

CREATE PROCEDURE GetStudentiByIdCorso(IN id INT)

BEGIN

    SELECT s.nome, s.cognome, s.email

    FROM Studenti AS s

    INNER JOIN Iscrizioni ON iscrizioni.id_studente=s.id_studente

    WHERE Iscrizioni.id_corso=id;

END //

DELIMITER ;

CALL GetStudentiByIdCorso(10);
```

2. Contare i corsi a cui è iscritto uno studente (parametro OUT)

```
DELIMITER //

CREATE PROCEDURE GetNumeroCorsiByIdStudente(IN id INT, OUT totale INT)

BEGIN

    SELECT COUNT(id_corso) INTO totale

    FROM Iscrizioni

    WHERE id_studente=id;

END //

DELIMITER ;

CALL GetNumeroCorsiByIdStudente(2, @totale);

SELECT @totale;
```

3. Aggiornare la città di residenza tramite email

```
DELIMITER //

CREATE PROCEDURE UpdateCitta(IN ema VARCHAR(100), IN cit VARCHAR(50))

BEGIN

    UPDATE Studenti SET citta=cit WHERE email=ema;
```

```
END //
```

```
DELIMITER ;
```

```
CALL UpdateCitta('andrea.fiore@studenti.it', 'Roma');
```

4. Verificare l'esistenza di un'email (con gestione errore)

```
DELIMITER //
```

```
CREATE PROCEDURE VerificaEmail(IN ema VARCHAR(100))
```

```
BEGIN
```

```
    DECLARE v_count INT DEFAULT 0;
```

```
    SELECT COUNT(*) INTO v_count
```

```
    FROM studenti
```

```
    WHERE email = ema;
```

```
    IF v_count > 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Email già esistente';
```

```
    END IF;
```

```
END //
```

```
DELIMITER ;
```

```
CALL VerificaEmail('daniele.orlando@studenti.it'); -> "ERROR 1644 (45000): Email già esistente".
```

```
CALL VerificaEmail('nuova.email@test.it'); -> viene eseguita correttamente senza errori.
```